

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ  
Національний університет «Полтавська політехніка  
імені Юрія Кондратюка»

Кафедра комп'ютерних та інформаційних технологій і систем

Звіт  
з лабораторної роботи №5  
з дисципліни «Технології на платформі NET»,  
тема: «Реалізація спроможностей реєстрації, аутентифікації та авторизації  
засобами фреймворку ASP.NET Core Identity.»

Виконав:

Студент групи 404-ТН

Плаксюк Я.М.

Перевірив:

Викладач каф. КІТС Тютюнник П.Б.

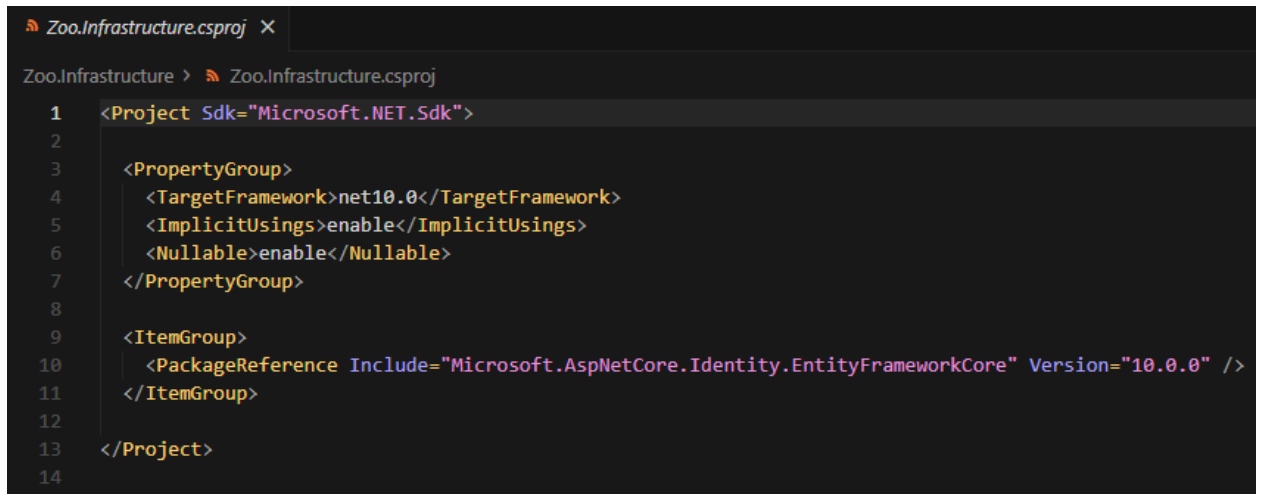
Полтава 2025

## Завдання

1. Додати в проєкт {Назва тематики}.Infrastructure нову залежність на NuGET пакет [Microsoft.AspNetCore.Identity.EntityFrameworkCore](#).
2. Модифікуйте створену вами у попередніх лабораторних роботах модель (набір класів), додавши до неї сутність користувача, яка буде наслідуватися від класу IdentityUser із фреймворку [ASP.NET Core Identity](#). Модифікуйте, створений у третій лабораторній, DbContext таким чином, щоб він наслідувався від класу IdentityDbContext<T>, де узагальнюючим типом буде створена вами сутність користувача.
3. Модифікуйте проєкт {Назва тематики}.REST, щоб він виставляв на зовні вбудовані у [ASP.NET Core Identity](#) кінцеві точки для реєстрації та аутентифікації, при цьому аутентифікація повинна створювати Bearer або JWT токени. Можете спиратися на [офіційний туторіал від Microsoft](#).
4. Модифікуйте створений у четвертій лабораторній REST API, щоб кінцеві точки використовували авторизацію, та надавали доступ відповідно до токена із заголовку HTTP-запиту Authorization. Зверніть увагу, щоб авторизація використовувалася лише в необхідних кінцевих точках, наприклад у REST API бібліотеки, перегляд книг не потребує авторизації, але бронювання книги потребує.
5. Додайте для створеного вебдодатку ролі, усього ролей повинно бути не менше трьох. Оновіть кінцеві точки REST API, що використовують авторизацію, щоб вони виконувалися залежно від ролі клієнта, наприклад у REST API бібліотеки, забронювати книгу може звичайний користувач, але змінити інформацію про книгу у БД лише бібліотекар.

## Виконання

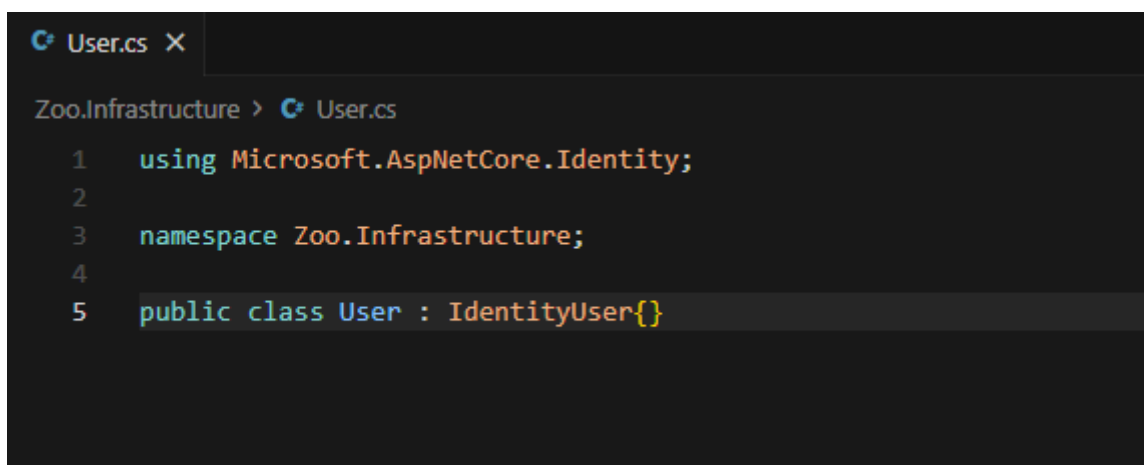
1. Додати в проєкт {Назва тематики}.Infrastructure нову залежність на NuGET пакет [Microsoft.AspNetCore.Identity.EntityFrameworkCore](#).



```
1 <Project Sdk="Microsoft.NET.Sdk">
2
3   <PropertyGroup>
4     <TargetFramework>net10.0</TargetFramework>
5     <ImplicitUsings>enable</ImplicitUsings>
6     <Nullable>enable</Nullable>
7   </PropertyGroup>
8
9   <ItemGroup>
10    <PackageReference Include="Microsoft.AspNetCore.Identity.EntityFrameworkCore" Version="10.0.0" />
11  </ItemGroup>
12
13 </Project>
14
```

Рис. 1 – Вигляд файлу Zoo.Infrastructure.csproj

2. Модифікуйте створену вами у попередніх лабораторних роботах модель (набір класів), додавши до неї сутність користувача, яка буде наслідуватися від класу IdentityUser із фреймворку [ASP.NET Core Identity](#). Модифікуйте, створений у третій лабораторній, DbContext таким чином, щоб він наслідувався від класу IdentityDbContext<T>, де узагальнюючим типом буде створена вами сутність користувача.



```
1 using Microsoft.AspNetCore.Identity;
2
3 namespace Zoo.Infrastructure;
4
5 public class User : IdentityUser{}
```

Рис. 2 – Доданий файл User.cs

```
ZooDbContext.cs X
Zoo.Infrastructure > ZooDbContext.cs
1  using Microsoft.AspNetCore.Identity.EntityFrameworkCore;
2  using Microsoft.EntityFrameworkCore;
3  using System;
4
5  namespace Zoo.Infrastructure;
6
7  public class ZooDbContext : IdentityDbContext<User>
8  {
9      public ZooDbContext(DbContextOptions<ZooDbContext> options)
10         : base(options)
11     {
12     }
13
14     public DbSet<Animal> Animals { get; set; }
15
16     protected override void OnModelCreating(ModelBuilder builder)
17     {
18         base.OnModelCreating(builder);
19
20         builder.Entity<Animal>(entity =>
21         {
22             entity.HasKey(e => e.Id);
23             entity.Property(e => e.Species).IsRequired();
24             entity.Property(e => e.Name).IsRequired();
25             entity.Property(e => e.Age).IsRequired();
26         });
27     }
28 }
29
30 public class Animal
31 {
32     public Guid Id { get; set; }
33     public string Species { get; set; } = string.Empty;
34     public string Name { get; set; } = string.Empty;
35     public int Age { get; set; }
36     public Guid EnclosureId { get; set; }
37 }
```

Рис. 3 – Доданий файл ZooDbContext.cs

3. Модифікуйте проєкт {Назва тематики}.REST, щоб він виставляв на зовні вбудовані у [ASP.NET](#) Core Identity кінцеві точки для реєстрації та аутентифікації, при цьому аутентифікація повинна створювати Bearer або JWT токени. Можете спиратися на [офіційний туторіал від Microsoft](#).

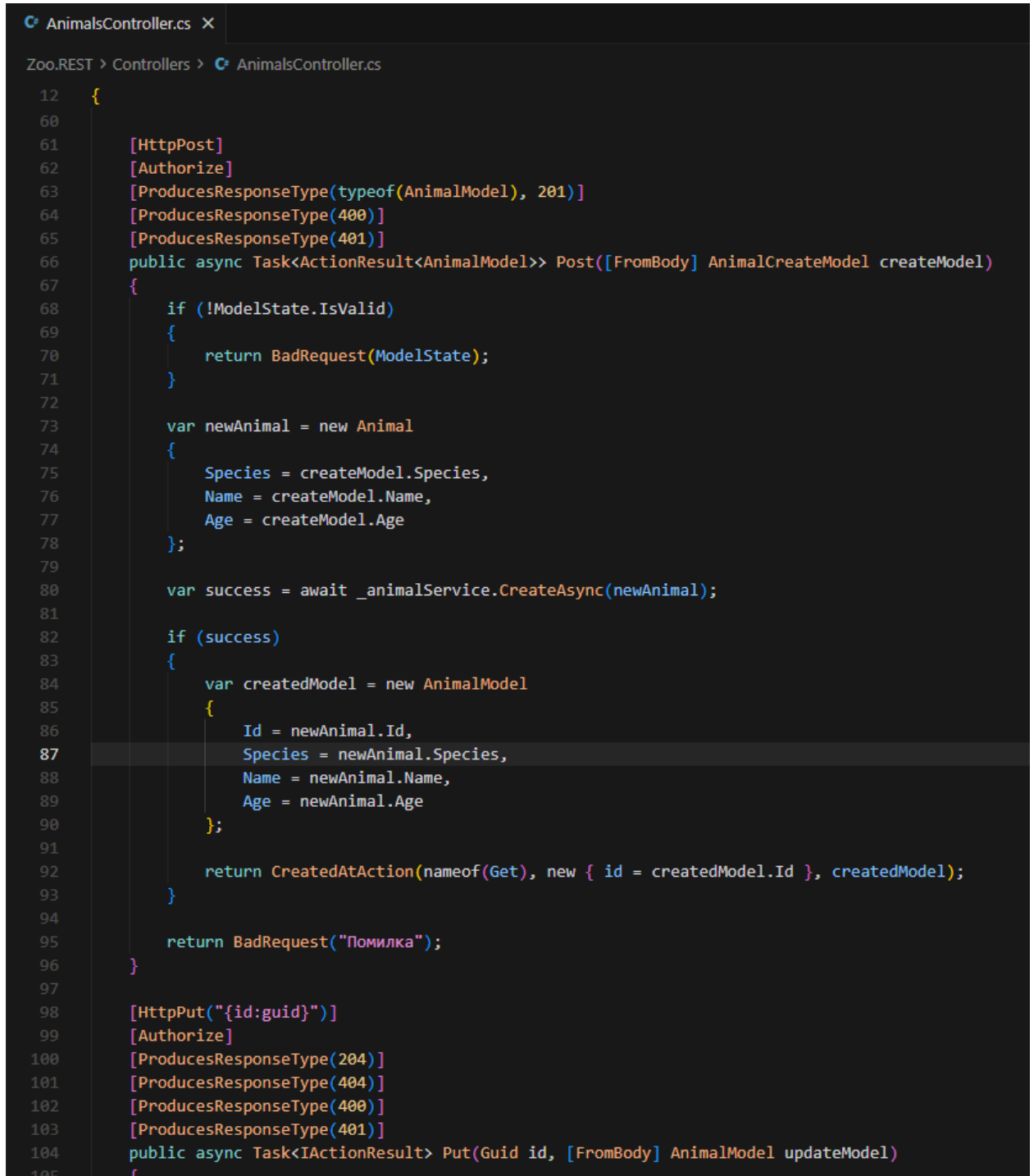
```
appsettings.json X
Zoo.REST > {} appsettings.json > ...
1 {
2   "Logging": {
3     "LogLevel": {
4       "Default": "Information",
5       "Microsoft.AspNetCore": "Warning"
6     }
7   },
8   "AllowedHosts": "*",
9   "ConnectionStrings": {
10    "DefaultConnection": "Server=(localdb)\\mssqllocaldb;Database=ZooDb;Trusted_Connection=True;MultipleActiveResultSets=true"
11  },
12  "Jwt": {
13    "Key": "YourSuperSecretKeyThatIsAtLeast32CharactersLong!",
14    "Issuer": "ZooSolution",
15    "Audience": "ZooSolution"
16  }
17 }
```

Рис. 4 – Вигляд файлу appsettings.json

```
Program.cs X
Zoo.REST > Program.cs
1 using Microsoft.AspNetCore.Authentication.Cookies;
2 using Microsoft.AspNetCore.Authentication.JwtBearer;
3 using Microsoft.AspNetCore.Identity;
4 using Microsoft.EntityFrameworkCore;
5 using Microsoft.IdentityModel.Tokens;
6 using System.Linq;
7 using System.Text;
8 using Zoo.Infrastructure;
9
10 var builder = WebApplication.CreateBuilder(args);
11
12 builder.Services.AddDbContext<ZooDbContext>(options =>
13     options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection")));
14
15 builder.Services.AddIdentityCore<User>(options =>
16 {
17     options.Password.RequireDigit = true;
18     options.Password.RequireLowercase = true;
19     options.Password.RequireUppercase = true;
20     options.Password.RequireNonAlphanumeric = false;
21     options.Password.RequiredLength = 6;
22     options.User.RequireUniqueEmail = true;
23 })
24 .AddRoles<IdentityRole>()
25 .AddEntityFrameworkStores<ZooDbContext>()
26 .AddSignInManager()
27 .AddApiEndpoints();
28
29 var jwtKey = builder.Configuration["Jwt:Key"] ?? throw new InvalidOperationException("JWT Key");
30 var jwtIssuer = builder.Configuration["Jwt:Issuer"] ?? throw new InvalidOperationException("JWT Issuer");
31 var jwtAudience = builder.Configuration["Jwt:Audience"] ?? throw new InvalidOperationException("JWT Audience");
32
33 builder.Services.AddAuthentication(options =>
34 {
35     options.DefaultAuthenticateScheme = JwtBearerDefaults.AuthenticationScheme;
36     options.DefaultChallengeScheme = JwtBearerDefaults.AuthenticationScheme;
37 })
38 .AddJwtBearer(options =>
39 {
40     options.TokenValidationParameters = new TokenValidationParameters
41     {
42         ValidateIssuer = true,
43         ValidateAudience = true,
44         ValidateLifetime = true,
```

Рис. 5 – Змінений файл Program.cs

4. Модифікуйте створений у четвертій лабораторній REST API, щоб кінцеві точки використовували авторизацію, та надавали доступ відповідно до токена із заголовку HTTP-запиту Authorization. Зверніть увагу, щоб авторизація використовувалася лише в необхідних кінцевих точках, наприклад у REST API бібліотеки, перегляд книг не потребує авторизації, але бронювання книги потребує.



```
12 {
60
61     [HttpPost]
62     [Authorize]
63     [ProducesResponseType(typeof(AnimalModel), 201)]
64     [ProducesResponseType(400)]
65     [ProducesResponseType(401)]
66     public async Task<ActionResult<AnimalModel>> Post([FromBody] AnimalCreateModel createModel)
67     {
68         if (!ModelState.IsValid)
69         {
70             return BadRequest(ModelState);
71         }
72
73         var newAnimal = new Animal
74         {
75             Species = createModel.Species,
76             Name = createModel.Name,
77             Age = createModel.Age
78         };
79
80         var success = await _animalService.CreateAsync(newAnimal);
81
82         if (success)
83         {
84             var createdModel = new AnimalModel
85             {
86                 Id = newAnimal.Id,
87                 Species = newAnimal.Species,
88                 Name = newAnimal.Name,
89                 Age = newAnimal.Age
90             };
91
92             return CreatedAtAction(nameof(Get), new { id = createdModel.Id }, createdModel);
93         }
94
95         return BadRequest("Помилка");
96     }
97
98     [HttpPut("{id:guid}")]
99     [Authorize]
100     [ProducesResponseType(204)]
101     [ProducesResponseType(404)]
102     [ProducesResponseType(400)]
103     [ProducesResponseType(401)]
104     public async Task<IActionResult> Put(Guid id, [FromBody] AnimalModel updateModel)
105     {
```

Рис. 6 – Змінений файл AnimalsController.cs

```

AuthController.cs X
Zoo.REST > Controllers > AuthController.cs
1  using Microsoft.AspNetCore.Authorization;
2  using Microsoft.AspNetCore.Identity;
3  using Microsoft.AspNetCore.Mvc;
4  using Microsoft.IdentityModel.Tokens;
5  using System.IdentityModel.Tokens.Jwt;
6  using System.Security.Claims;
7  using System.Text;
8  using Zoo.Infrastructure;
9
10 namespace Zoo.REST.Controllers;
11
12 [ApiController]
13 [Route("api/[controller]")]
14 public class AuthController : ControllerBase
15 {
16     private readonly UserManager<User> _userManager;
17     private readonly SignInManager<User> _signInManager;
18     private readonly IConfiguration _configuration;
19
20     public AuthController(
21         UserManager<User> userManager,
22         SignInManager<User> signInManager,
23         IConfiguration configuration)
24     {
25         _userManager = userManager;
26         _signInManager = signInManager;
27         _configuration = configuration;
28     }
29
30     [HttpPost("login")]
31     [AllowAnonymous]
32     public async Task<IActionResult> Login([FromBody] LoginRequest request)
33     {
34         var user = await _userManager.FindByEmailAsync(request.Email);
35         if (user == null)
36         {
37             return Unauthorized(new { message = "Неправильна електронна адреса або пароль" });
38         }
39
40         var result = await _signInManager.CheckPasswordSignInAsync(user, request.Password, lockoutOnFailure: false);
41         if (!result.Succeeded)
42         {
43             return Unauthorized(new { message = "Неправильна електронна адреса або пароль" });
44         }
45
46         var roles = await _userManager.GetRolesAsync(user);
47         var claims = new List<Claim>

```

Рис. 7 – Доданий файл AuthController.cs

5. Додайте для створеного вебдодатку ролі, усього ролей повино бути не менше трьох. Оновіть кінцеві точки REST API, що використовують авторизацію, щоб вони виконувалися залежно від ролі клієнта, наприклад у REST API бібліотеки, забронювати книгу може звичайний користувач, але змінити інформацію про книгу у БД лише бібліотекар.

```
Program.cs X
Zoo.REST > Program.cs
39 {
40     {
41         validateIssuer = true,
42         ValidateAudience = true,
43         ValidateLifetime = true,
44         ValidateIssuerSigningKey = true,
45         ValidIssuer = jwtIssuer,
46         ValidAudience = jwtAudience,
47         IssuerSigningKey = new SymmetricSecurityKey(Encoding.UTF8.GetBytes(jwtKey)),
48         RoleClaimType = "http://schemas.microsoft.com/ws/2008/06/identity/claims/role"
49     };
50 });
51 .AddCookie("Identity.Bearer", options =>
52 {
53     options.Cookie.Name = "Identity.Bearer";
54     options.Cookie.HttpOnly = true;
55     options.Cookie.SameSite = SameSiteMode.Lax;
56     options.Cookie.SecurePolicy = CookieSecurePolicy.SameAsRequest;
57     options.ExpireTimeSpan = TimeSpan.FromHours(1);
58 });
59 builder.Services.AddAuthorization();
60 builder.Services.AddControllers();
61 builder.Services.AddEndpointsApiExplorer();
62 builder.Services.AddSingleton<ICrudServiceAsync<Animal>, AnimalService>();
63 builder.Services.AddOpenApi();
64 var app = builder.Build();
65 using (var scope = app.Services.CreateScope())
66 {
67     var dbContext = scope.ServiceProvider.GetRequiredService<ZooDbContext>();
68     dbContext.Database.EnsureCreated();
69     var roleManager = scope.ServiceProvider.GetRequiredService<RoleManager<IdentityRole>>();
70     var roles = new[] { "Visitor", "User", "Zookeeper", "Admin" };
71     foreach (var role in roles)
72     {
73         if (!await roleManager.RoleExistsAsync(role))
74         {
75             await roleManager.CreateAsync(new IdentityRole(role));
76         }
77     }
78 }
```

Рис. 8 – Змінений файл Program.cs



```

AnimalsController.cs X
Zoo.REST > Controllers > AnimalsController.cs
12 {
61 [HttpPost]
62 [Authorize(Roles = "User,AA,Admin")]
63 [ProducesResponseType(typeof(AnimalModel), 201)]
64 [ProducesResponseType(400)]
65 [ProducesResponseType(401)]
66 [ProducesResponseType(403)]
67 public async Task<ActionResult<AnimalModel>> Post([FromBody] AnimalCreateModel createModel)
68 {
69     if (!ModelState.IsValid)
70     {
71         return BadRequest(ModelState);
72     }
73
74     var newAnimal = new Animal
75     {
76         Species = createModel.Species,
77         Name = createModel.Name,
78         Age = createModel.Age
79     };
80
81     var success = await _animalService.CreateAsync(newAnimal);
82
83     if (success)
84     {
85         var createdModel = new AnimalModel
86         {
87             Id = newAnimal.Id,
88             Species = newAnimal.Species,
89             Name = newAnimal.Name,
90             Age = newAnimal.Age
91         };
92
93         return CreatedAtAction(nameof(Get), new { id = createdModel.Id }, createdModel);
94     }
95
96     return BadRequest("Помилка");
97 }
98
99 [HttpPut("{id:guid}")]
100 [Authorize(Roles = "BB,Admin")]
101 [ProducesResponseType(204)]
102 [ProducesResponseType(404)]
103 [ProducesResponseType(400)]
104 [ProducesResponseType(401)]
105 [ProducesResponseType(403)]
106 public async Task<IActionResult> Put(Guid id, [FromBody] AnimalModel updateModel)
107 {

```

Рис. 9 – Змінений файл AnimalsController.cs

## Тестування у Postman

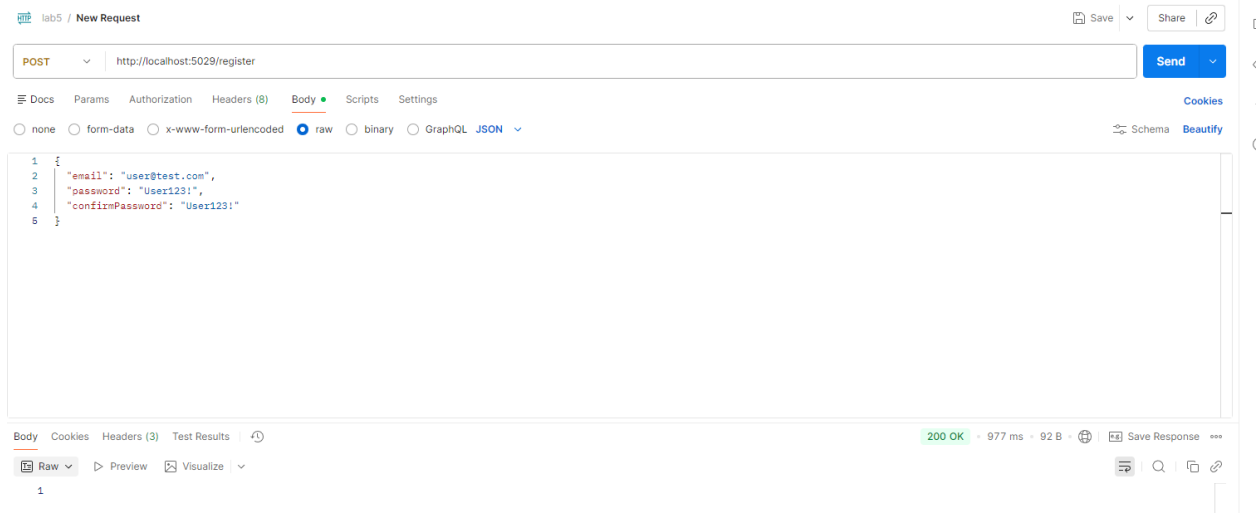


Рис. 10 – Реєстрація користувача

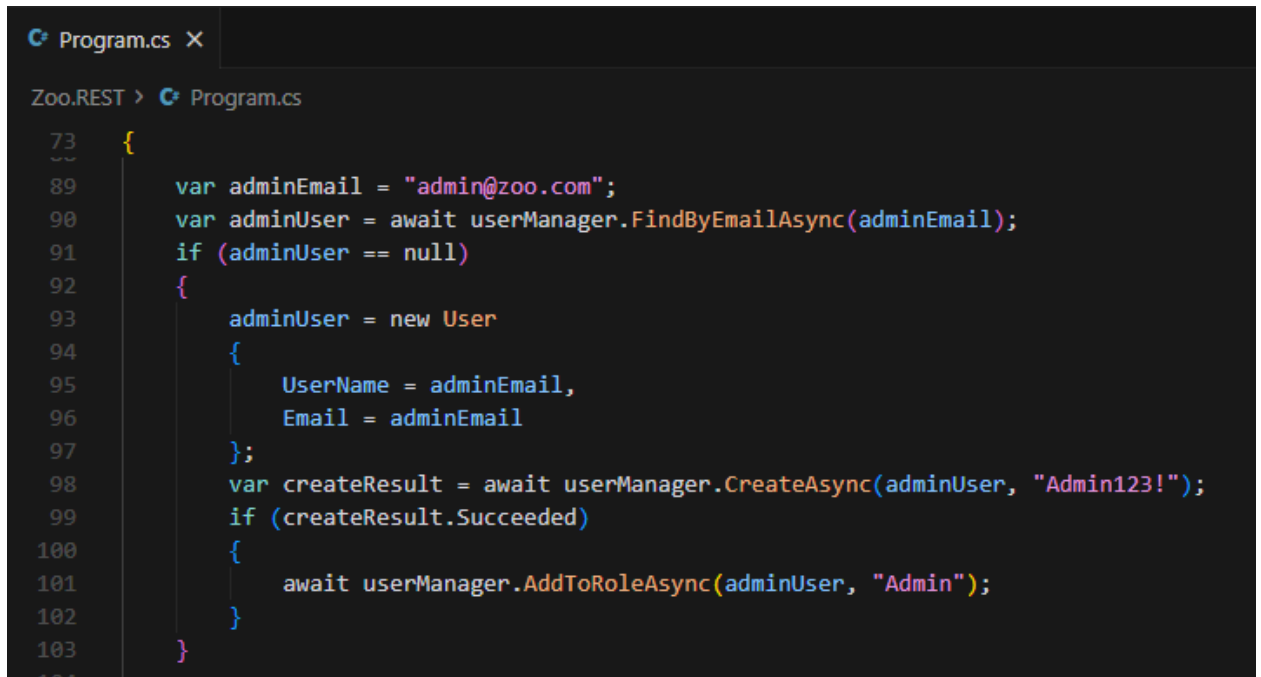


Рис. 11 – Адмін створився автоматично (бо один адмін повинен існувати)



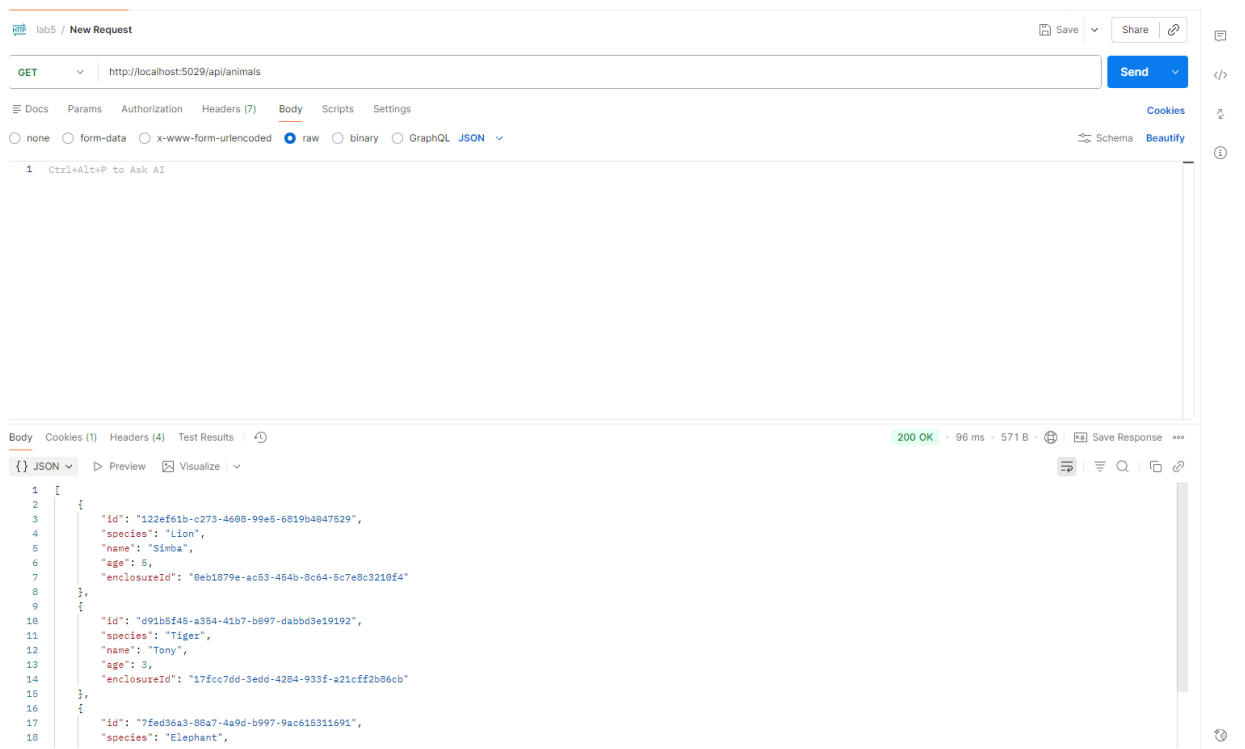


Рис. 14 – Отримання всіх тварин (Публічний доступ)

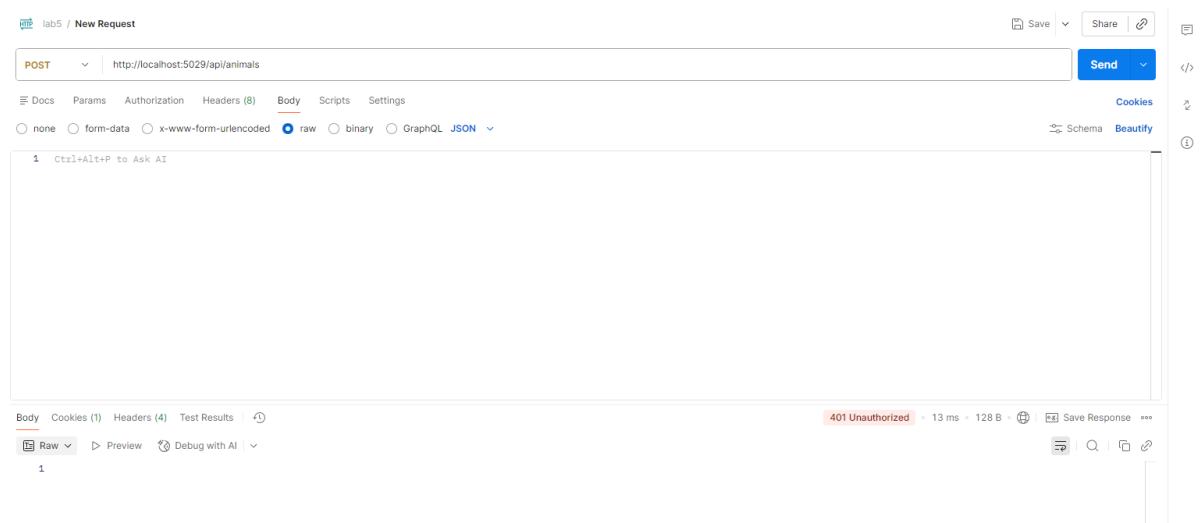


Рис. 15 – Отримання всіх тварин (Неавторизованість)

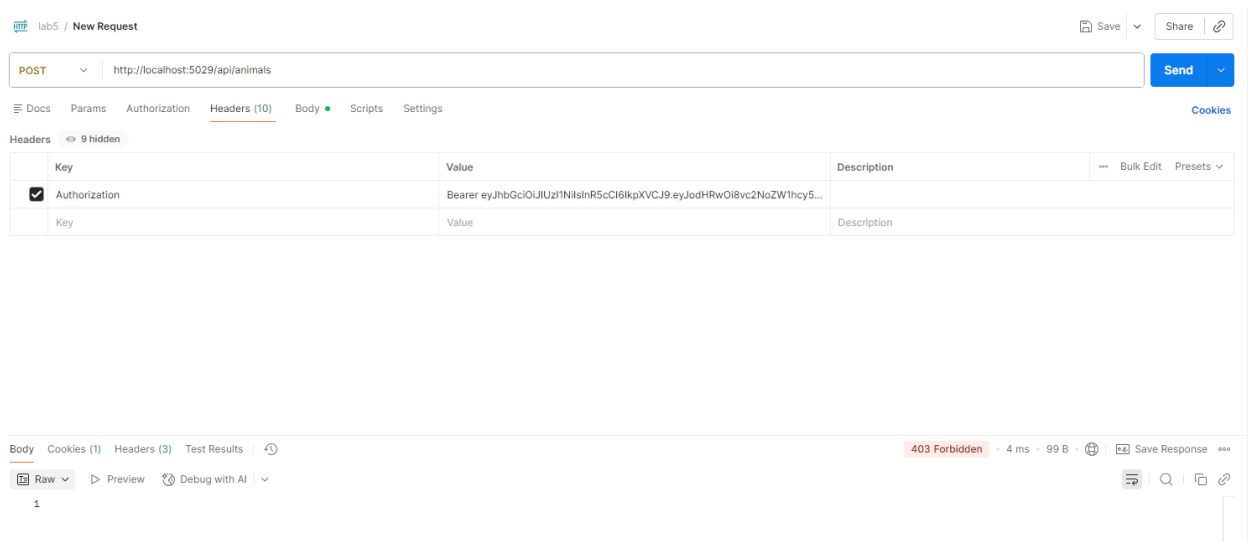


Рис. 16 – Додавання токена користувача та спроба додавання, але помилка 403, бо прав недостатньо у користувача

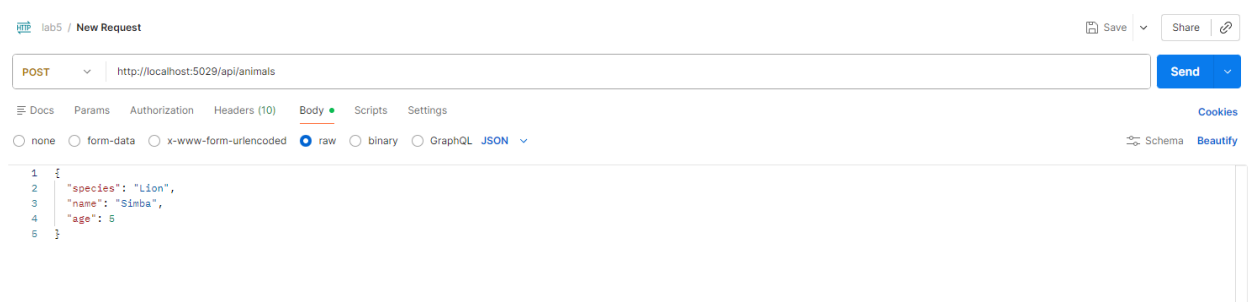


Рис. 17 – Тіло запиту

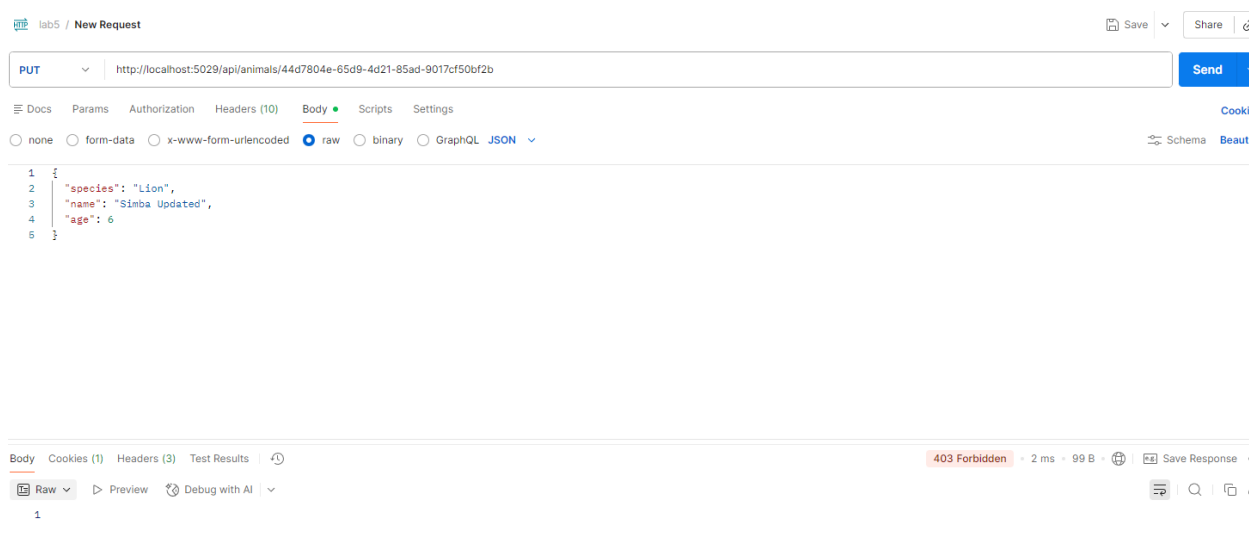


Рис. 18 – Спроба оновлення, але помилка 403, бо прав недостатньо у користувача

## **Висновок**

У ході лабораторної роботи я додав до існуючої лб4, авторизацію користувача. Була додана папка Zoo.Infrastructure – воно як контекстна база даних. Потім були змінені відповідні контролери та доданий новий. Результати запитів були отримані у Postman, щоб наочно продемонструвати реєстрацію та доступ.